Package 'levyBridge'

October 27, 2026

Title Stochastic Bridges for Levy Degradation Processes

Version 0.1.0

Description Exact simulation of the bridges of Levy degradation processes (gamma, Wiener, inverse Gaussian, variance-gamma and Meixner processes) and resampling of trajectories observed at irregular inspection times onto a regular time grid, following the interpolation algorithm of Grall-Maes (2012) <doi:10.1177/1748006X12462522>. The reconstructed increments are the basis of goodness-of-fit tests.

License GPL-3

URL https://github.com/rytracetito/levy_packages/tree/main/levyBridge, https://github.com/rytracetito/levy_packages

BugReports https://github.com/rytracetito/levy_packages/issues

Encoding UTF-8

Language en

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Suggests DegradSim, levyfit, statmod, testthat (>= 3.0.0)

Config/testthat/edition 3

Imports ggplot2, GIGrvg, gsl, Runuran, stats, utils

Author Amedée Rytrace Tito [aut, cre], Christian Paroissin [aut]

Maintainer Amedée Rytrace Tito <rytracetito5@gmail.com>

Contents

levyBridge-package 2
gamma_bridge 5
ig_bridge 11
increments 16
meixner_bridge 18
plot_bridge 23
vg_bridge 24
wiener_bridge 30

Index **35**

`levyBridge-package` *levyBridge: stochastic bridges for Levy degradation processes*

Description

`levyBridge` resamples degradation trajectories observed at irregular inspection times onto a regular time grid, by exact simulation of the bridge of the process between the known values. Under the model, the reconstructed increments are independent and identically distributed: they are the input of the goodness-of-fit tests for Levy degradation processes.

Bridge of a Levy process

Let X be a Levy process such that X_s has the density f_s, and $t_A < t < t_B$. Given $X_{t_A} = x_A$ and $X_{t_B} = x_B$, X_t has the density

$$f_{t|t_A,t_B}(x) = \frac{f_{t-t_A}(x - x_A)\, f_{t_B-t}(x_B - x)}{f_{t_B-t_A}(x_B - x_A)}$$

(Hoyle, Hughston and Macrina, 2011). Each `_bridge` function draws exactly from this law, for one process:

Function	Process	Parameters	Simulation of the bridge
`gamma_bridge()`	gamma	shape `a`	beta variable
`wiener_bridge()`	Wiener	`sigma`	Gaussian variable
`ig_bridge()`	inverse Gaussian	`eta`	IG or scaled inverse IG
`vg_bridge()`	variance-gamma	`theta`, `sigma`, `nu`	GIG, beta, Gaussian
`meixner_bridge()`	Meixner	`a`, `d`, `m`	rejection, Cauchy envelope

The law of a bridge never depends on all the parameters of the process: the gamma bridge does not depend on the rate `b`, the Brownian bridge does not depend on the drift `mu`, the inverse Gaussian bridge does not depend on `mu`, and the Meixner bridge does not depend on `b`. Click on a function name for the law of its bridge and the details of the simulation.

Resampling methods

Every `_bridge` function has an argument `method`:

- `"Bridge"` (default): interpolation algorithm of Grall-Maës (2012); see also the help page of `gamma_bridge()`. A trajectory observed at $0 = t_0 < t_1 < \cdots < t_N$ is resampled on the regular grid $t'_k = k\, t_N/N$, $k = 0, \ldots, N$: as many points as observations, with the time step $\Delta t_{eq} = t_N/N$. The grid is covered from left to right; each grid point is drawn from the bridge between the last known value before it (an observation or the previous grid point, already drawn) and the next observation. A grid point equal to an inspection time keeps the observed value;
- `"ExtBridge"`: extension of this method whose grid contains all the inspection times. With d the number of decimal places of the times (argument `digits`, found automatically by default), the time step is $h = 10^{-d} \gcd(10^d t_1, \ldots, 10^d t_N)$, the largest one such that every inspection time is a grid point. The grid is covered as for `"Bridge"`, so the pseudo-sample contains all the observations. The grid has $t_N/h \geq N$ steps, with equality for regular times. With several trajectories, `common_step = TRUE` gives them the same step h (gcd of all the gaps of all the trajectories);

- `"MultiBridge"`: the method `"Bridge"` for several trajectories, on one regular grid common to all of them, of step $h = (t_{1,max} + \cdots + t_{n,max})/(m_1 + \cdots + m_n)$ (total observed duration divided by the total number of increments; t_N/N for one trajectory). With `option = 1` (default) each trajectory is resampled up to its own last inspection; with `option = 2` all the trajectories go up to the last inspection time over all of them, the shorter ones being extended by increments of the process (this needs all the parameters, and these extended values do not come from the data).

The three methods use the same bridges and the same algorithm: only the grid differs.

Which method?

- one trajectory: `"Bridge"`, or `"ExtBridge"` to keep all the observations on the grid;
- several trajectories, one test per trajectory: `"Bridge"` or `"ExtBridge"`;
- several trajectories, one test on all of them together: the reconstructed increments must have the same law, hence the same time step: `"MultiBridge"` (grid of Grall-Maës type, about as many points as observations) or `"ExtBridge"` with `common_step = TRUE` (grid containing all the inspection times, often many more points).

Data

All the `_bridge` functions accept the same formats:

- **one trajectory**: a vector of levels `x` and a vector of inspection times `t`, starting at 0, or a data frame whose first column contains the times and second column the levels;
- **several trajectories with the same inspection times**: the matrix returned by the `DegradSim` simulation functions, with columns `t`, `X1`, `X2`, ...;
- **several trajectories, each with its own inspection times**: a list of data frames, one per trajectory (first column the times, second column the levels), for instance the list returned by `DegradSim` with one plan per trajectory.

With the methods "Bridge" and "ExtBridge", each trajectory gets its own regular grid and its own time step. With the method "MultiBridge", or "ExtBridge" with `common_step = TRUE`, all the trajectories get the same time step: their reconstructed increments then have the same law and can be tested together.

Results

A `_bridge` function returns an object of class `"levybridge"`:

- `res` prints the model, the parameters, the time steps and the first resampled values;
- `res[[k]]` is the k-th trajectory on its regular grid, a data frame with columns `t` and `X`;
- `attr(res, "teq")` gives the time step of the grid of each trajectory;
- `increments()` gives the reconstructed increments, one row per increment with the columns `id` (trajectory), `teq` and `dx`: they are the values to be tested;
- `plot_bridge()` compares the observed and resampled values (`ggplot2`).

The bridge values are random: use `set.seed()` before a resampling to make it reproducible.

Goodness-of-fit test, step by step

The package covers steps 2 to 4 of the methodology of goodness-of-fit tests for a parametric family of Levy processes:

1. **Estimate** the parameters from the observed trajectories, for instance with the package `levyfit` (`gamma_mle()`, `meixner_mle()`, ...);
2. **Law of the bridge**: it is given in the help page of each `_bridge` function;
3. **Simulation of the bridge**: done by the `_bridge` function, with the estimated parameters;
4. **Pseudo-sample on a regular grid**: the result of the `_bridge` function, with the method "Bridge", "ExtBridge" or "MultiBridge";
5. **Choose a test** for an i.i.d. sample of the law of $X_{\Delta t_{eq}}$, for instance Kolmogorov-Smirnov, Cramér-von Mises or Anderson-Darling;
6. **Apply the test** to the reconstructed increments `increments(res)$dx`.

Author(s)

Maintainer: Amedée Rytrace Tito <rytracetito5@gmail.com>

Authors:

- Christian Paroissin <christian.paroissin@univ-pau.fr>

References

Grall-Maës, E. (2012). Use of the Kolmogorov-Smirnov test for gamma process. *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability*, 226(6), 624-634. doi:10.1177/1748006X12462522

Hoyle, E., Hughston, L. P. and Macrina, A. (2011). Lévy random bridges and the modelling of financial information. *Stochastic Processes and their Applications*, 121(4), 856-884.

Tito, A. R. and Paroissin, C. (in preparation).

See Also

Useful links:

- https://github.com/rytracetito/levy_packages/tree/main/levyBridge
- https://github.com/rytracetito/levy_packages
- Report bugs at https://github.com/rytracetito/levy_packages/issues

Examples

```
# 1. One gamma trajectory observed at irregular times
t <- c(0, 1.2, 2, 3.8, 5.5, 7.1, 10.4, 12.9, 15.3, 18.7, 22.5)
set.seed(1)
x <- c(0, cumsum(rgamma(10, shape = 2 * diff(t), rate = 3)))

# Resampling on the regular grid 0, 2.25, 4.5, ..., 22.5
r <- gamma_bridge(x, t, a = 2)
r
r[[1]]
attr(r, "teq")
increments(r)
plot_bridge(r)
```

```
# 2. The same trajectory with the other processes (parameters of the bridge only)
wiener_bridge(x, t, sigma = 1.5)[[1]]
ig_bridge(x, t, eta = 3)[[1]]
vg_bridge(x, t, theta = 0.5, sigma = 1.5, nu = 2)[[1]]
meixner_bridge(x, t, a = 1, d = 1.5, m = 0.2)[[1]]

# 3. Several trajectories, each with its own inspection times (a list)
liste <- list(data.frame(t = c(0, 1, 3, 6), X = c(0, 0.4, 2.3, 2.9)),
              data.frame(t = c(0, 2, 4, 5, 9), X = c(0, 1.6, 2.8, 3.1, 4.2)))
r2 <- gamma_bridge(liste, a = 2)
attr(r2, "teq")
increments(r2)

# 4. Complete use: simulation (DegradSim), estimation (levyfit), bridge
plan <- DegradSim::inspection_times("increasing", n = 20, n_traj = 10, tmax = 40)
sims <- DegradSim::sim_gamma(plan, a = 2, b = 3)

# Step 1: estimation of the parameters
est <- levyfit::gamma_mle(sims)
est

# Steps 3 and 4: resampling with the estimated shape
r3 <- gamma_bridge(sims, a = est["a"])
plot_bridge(r3, trajectory = 1:4)

# Step 6: the reconstructed increments, to be tested against the law
# Gamma(a teq, b) with the estimated parameters
d <- increments(r3)
head(d)
```

---

gamma_bridge *Gamma bridge: resampling on a regular grid*

Description

Resamples one or several trajectories of a gamma process, observed at irregular inspection times, onto a regular time grid, by exact simulation of the gamma bridge between the known values.

Usage

```
gamma_bridge(
  x,
  t = NULL,
  a,
  method = "Bridge",
  digits = NULL,
  common_step = FALSE,
  option = 1,
  b = NULL
)
```

Arguments

x	Degradation levels of one trajectory; or a list of data frames, one per trajectory, whose first column contains the inspection times and second column the degradation levels (for instance the result of a DegradSim simulation function).
t	Inspection times of the trajectory, starting at 0 and strictly increasing. Not used when x is a list.
a	Shape parameter of the gamma process, $a > 0$, for instance estimated with the function gamma_mle() of the package levyfit.
method	"Bridge" (default) for the algorithm of Grall-Maës (2012), with as many grid points as observations and one grid per trajectory; "ExtBridge" for its extension, whose grid contains all the inspection times (see the section Method "ExtBridge"); "MultiBridge" for several trajectories on one common grid (see the section Method "MultiBridge").
digits	Used only with method = "ExtBridge": number of decimal places of the inspection times, which sets the time step of the grid. NULL (default) finds it automatically.
common_step	Used only with method = "ExtBridge" and several trajectories: FALSE (default) gives each trajectory its own time step h; TRUE gives all the trajectories the same time step, the largest one such that every inspection time of every trajectory is a grid point. With a common step, the reconstructed increments of all the trajectories have the same law under the model and can be tested together.
option	Used only with method = "MultiBridge": 1 (default) to resample each trajectory up to its own last inspection, 2 to resample all the trajectories up to the last inspection time over all the trajectories, extending the shorter ones by increments of the process (see the section Method "MultiBridge").
b	Rate parameter of the gamma process, $b > 0$; used only by the option 2 of the method "MultiBridge", to extend the trajectories.

Value

An object of class "levybridge": a list with one data frame per trajectory, with columns t (regular grid) and X (resampled levels). Its attributes are model, parameters, method, teq (time step of the grid of each trajectory) and observed (the observed data, one data frame per trajectory; with the method "ExtBridge", the times are the rounded ones when a rounding was needed). Use increments() for the reconstructed increments and plot_bridge() to compare observed and resampled values.

Model

The gamma process with shape $a > 0$ and rate $b > 0$ has $X_0 = 0$, independent increments and $X_t - X_s \sim \Gamma(a(t-s), b)$ for $s < t$ (shape $a(t-s)$, rate b). Its trajectories are increasing.

Law of the bridge

Let $t_A < t < t_B$, $\tau_1 = t - t_A$, $\tau_2 = t_B - t$ and $\Delta = x_B - x_A$. Given $X_{t_A} = x_A$ and $X_{t_B} = x_B$,

$$X_t = x_A + \Delta\, Y, \qquad Y \sim \mathrm{Beta}(a\tau_1, a\tau_2).$$

The law of the bridge does not depend on the rate b: only the shape a is needed. When $\Delta = 0$ (two equal observed values, which can happen after rounding), the bridge is constant.

Resampling algorithm (method "Bridge")

This is the interpolation algorithm of Grall-Maës (2012), used for all the processes of the package. Let $0 = t_0 < t_1 < \cdots < t_N$ be the inspection times of a trajectory and $x_0, \ldots, x_N$ the observed levels. The regular grid is $t'_i = i\,\Delta t_{eq}$, $i = 0, \ldots, N$, with $\Delta t_{eq} = t_N/N$: it has as many points as observations and the same end points.

1. $x'_0 = x_0$ and $x'_N = x_N$.
2. For $i = 1, \ldots, N-1$, in increasing order:
   - if t'_i is an inspection time, x'_i is the observed value;
   - otherwise, x'_i is drawn from the bridge between the nearest known point on the left (an inspection or the grid point t'_{i-1} already drawn, whichever is closer) and the nearest inspection on the right.

By the Markov property of the bridge, the vector $(x'_1, \ldots, x'_{N-1})$ follows exactly the conditional law of $(X_{t'_1}, \ldots, X_{t'_{N-1}})$ given the observations. Hence, if the data come from the model and the parameters are the true ones, the reconstructed increments $x'_i - x'_{i-1}$ (see `increments()`) are independent with the law of $X_{\Delta t_{eq}}$: they can be tested with an i.i.d. goodness-of-fit test. A grid point that equals an inspection time up to rounding errors (relative difference below 10^{-9}) is set to that inspection time.

Precision. Each draw splits the increment Δ of the current interval into a left part (up to the grid point) and a right part (up to the next inspection), both computed directly: here ΔY and $\Delta(1-Y)$, with $Y = G_1/(G_1+G_2)$, $G_1 \sim \Gamma(a\tau_1)$, $G_2 \sim \Gamma(a\tau_2)$ drawn on the log scale. The reconstructed increments (`increments()`) are these parts, not differences of levels: a very small increment keeps all its significant digits instead of being rounded to 0, even for very small shapes $a\tau$; only an increment below the smallest positive number (about 10^{-308}), which needs extremely small shapes, is 0.

Method "ExtBridge"

Extension of the method of Grall-Maës whose regular grid contains all the inspection times (Tito and Paroissin). The inspection times are multiples of 10^{-d}, where d is the number of decimal places (argument `digits`), and the time step is the largest one such that every inspection time is a grid point:

$$h = 10^{-d}\,\gcd\!\left(10^d t_1, \ldots, 10^d t_N\right), \qquad t'_k = k\,h, \quad k = 0, \ldots, t_N/h.$$

The grid is then covered as in the method "Bridge": every inspection time keeps its observed value, and the other grid points are drawn from the bridge between the last known value on the left and the next inspection. The reconstructed increments are independent with the law of X_h, and the pseudo-sample contains all the observations. The grid has $t_N/h \geq N$ steps, with equality if and only if the inspection times are regular; then the method returns the observations themselves.

With `digits = NULL` (default), d is the smallest number of decimal places, between 0 and 6, such that all the inspection times are multiples of 10^{-d}. If there is none (for instance times drawn with `runif()`), the times are rounded to 2 decimal places, with a warning; give `digits` to choose another rounding. When `digits` is given and some times have more decimal places, they are rounded to `digits` decimal places, with a warning; two times equal after rounding give an error. A warning is also given when the grid has more than 10^5 steps, since the resampling then takes time. The grid and the time step are computed for each trajectory; with `common_step = TRUE`, d and h are computed from the inspection times of all the trajectories, so that all the grids have the same step.

Method "MultiBridge"

Version of the method "Bridge" for several trajectories, with **one regular grid common to all the trajectories** (Tito and Paroissin). With the method "Bridge", each trajectory has its own time step t_N/N: the reconstructed increments of two trajectories then have different laws and cannot be tested together. With the method "MultiBridge", all the reconstructed increments have the law of X_h for the same h: they form a single i.i.d. sample under the model, and one goodness-of-fit test can be applied to all the trajectories together.

Time step. For n trajectories, the trajectory i being observed on $[0, t_{i,max}]$ with m_i increments,

$$h = \frac{t_{1,max} + \cdots + t_{n,max}}{m_1 + \cdots + m_n},$$

the total observed duration divided by the total number of increments. The number of reconstructed increments is then about the number of observed increments, as with the method "Bridge"; for one trajectory, $h = t_N/N$ is the step of the method "Bridge". The grid is $t'_k = kh$; as with the method "Bridge", it does not necessarily contain the inspection times (a grid point equal to an inspection time keeps the observed value).

Option 1 (`option = 1`, default): each trajectory is resampled up to its own last inspection, on the grid points $kh \leq t_{i,max}$. Each grid point is drawn from the bridge between the last known value on the left and the next inspection, exactly as with the method "Bridge". The end of the trajectory, between its last grid point and $t_{i,max}$, is not used. A long trajectory gives many reconstructed increments, a short one few (none when $t_{i,max} < h$, with a warning). Under the model, with the true parameters, all the reconstructed increments are independent with the law of X_h: the pooled sample is exactly i.i.d. This is the recommended option.

Option 2 (`option = 2`): all the trajectories are resampled up to the last inspection time over all the trajectories, $\max_i t_{i,max}$, so that they all have the same number of grid points. Up to $t_{i,max}$, as in option 1; beyond, there is no observation left and the trajectory is extended by increments of the process drawn with the given parameters (not a bridge): the first one starts from the last observation $x_{i,N}$. This needs all the parameters of the process (for instance b for the gamma process, see the arguments). Under the model, with the true parameters, the pooled sample is also i.i.d. with the law of X_h; but the extended values do not come from the data: they follow the null model by construction and dilute the information of the observations, which lowers the power of a test, all the more as the trajectories have different durations. Use it only for trajectories of similar durations.

When to use "MultiBridge". For several trajectories observed at different times, when one test on all the trajectories together is wanted. With one trajectory, both options give the method "Bridge". The method "ExtBridge" with `common_step = TRUE` also gives a common step, with a grid that contains all the inspection times, but it can have many more grid points.

References

Grall-Maës, E. (2012). Use of the Kolmogorov-Smirnov test for gamma process. *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability*, 226(6), 624-634. doi:10.1177/1748006X12462522

Avramidis, A. N., L'Ecuyer, P. and Tremblay, P.-A. (2003). Efficient simulation of gamma and variance-gamma processes. *Proceedings of the 2003 Winter Simulation Conference*, 319-326.

See Also

- `increments()`: the reconstructed increments;
- `plot_bridge()`: observed and resampled values;
- `wiener_bridge()`: bridge of the Wiener process;

- ig_bridge(): bridge of the inverse Gaussian process;
- vg_bridge(): bridge of the variance-gamma process;
- meixner_bridge(): bridge of the Meixner process.

Examples

```
# 1. One trajectory observed at 11 irregular times (a = 2, b = 3)
t <- c(0, 1.2, 2, 3.8, 5.5, 7.1, 10.4, 12.9, 15.3, 18.7, 22.5)
set.seed(1)
x <- c(0, cumsum(rgamma(10, shape = 2 * diff(t), rate = 3)))
set.seed(1)
r <- gamma_bridge(x, t, a = 2)
r

# The resampled trajectory, on the regular grid 0, 2.25, 4.5, ..., 22.5
r[[1]]

# Time step of the grid: t_N / N = 22.5 / 10
attr(r, "teq")

# 2. Reconstructed increments, the input of the goodness-of-fit tests
increments(r)

# 3. Observed and resampled values
plot_bridge(r)

# 4. Grid points equal to inspection times keep the observed values:
#    here the grid is 0, 2, 4, 6, 8, 10 and 2 and 6 are inspection times
t2 <- c(0, 2, 3, 6, 7, 10)
x2 <- c(0, 1.4, 2.0, 4.1, 4.6, 6.8)
gamma_bridge(x2, t2, a = 2)[[1]]

# 5. One trajectory given as a data frame (times, then levels)
gamma_bridge(data.frame(t = t2, X = x2), a = 2)

# 6. Several trajectories, each with its own inspection times:
#    one regular grid and one time step per trajectory
liste <- list(data.frame(t = c(0, 1, 3, 6), X = c(0, 0.8, 2.1, 4.3)),
              data.frame(t = c(0, 2, 4, 5, 9), X = c(0, 1.2, 2.9, 3.1, 6.2)))
r2 <- gamma_bridge(liste, a = 2)
r2
attr(r2, "teq")
increments(r2)
plot_bridge(r2, trajectory = 1:2)

# 7. The bridge values are random: set.seed() makes them reproducible
set.seed(2); y1 <- gamma_bridge(x, t, a = 2)[[1]]$X
set.seed(2); y2 <- gamma_bridge(x, t, a = 2)[[1]]$X
identical(y1, y2)

# 8. Equal observed values (after rounding) give a constant bridge,
#    a decrease is impossible for this increasing process
gamma_bridge(c(0, 1, 1, 2), c(0, 1, 2.5, 3), a = 2)[[1]]
try(gamma_bridge(c(0, 2, 1.5, 3), c(0, 1, 2, 3), a = 2))

# 9. Method "ExtBridge": the times have 1 decimal place, the gcd of the
```

```
#    gaps is 0.1, and the grid 0, 0.1, ..., 22.5 contains all of them
r3 <- gamma_bridge(x, t, a = 2, method = "ExtBridge")
r3
attr(r3, "teq")
nrow(r3[[1]])
all(t %in% r3[[1]]$t)
plot_bridge(r3)

# Times with 2 decimal places give a grid of step 0.01, here 0.05
t4 <- c(0, 0.35, 1.2, 1.85, 2.5)
x4 <- c(0, 0.3, 1.1, 1.4, 2.2)
attr(gamma_bridge(x4, t4, a = 2, method = "ExtBridge"), "teq")

# digits = 1 rounds the times to 1 decimal place, with a warning
attr(gamma_bridge(x4, t4, a = 2, method = "ExtBridge", digits = 1), "teq")

# Regular times: the grid is the inspection times, the data are unchanged
gamma_bridge(c(0, 1.2, 2.9, 3.5), c(0, 2, 4, 6), a = 2, method = "ExtBridge")[[1]]

# Several trajectories with a common step: gcd of all the gaps, here 0.25
liste2 <- list(data.frame(t = c(0, 1, 3, 6), X = c(0, 0.8, 2.1, 4.3)),
               data.frame(t = c(0, 0.5, 1.25, 2), X = c(0, 0.4, 1.1, 1.6)))
attr(gamma_bridge(liste2, a = 2, method = "ExtBridge"), "teq")
attr(gamma_bridge(liste2, a = 2, method = "ExtBridge", common_step = TRUE), "teq")

# Small shapes: the reconstructed increments can be tiny, but they are not rounded to 0
min(increments(gamma_bridge(x, t, a = 0.5, method = "ExtBridge"))$dx)

# Method "MultiBridge": four trajectories with their own times, one common grid
liste_t <- list(c(0, 3, 8), c(0, 2, 5, 9, 12), c(0, 4, 7, 11, 15, 18, 22, 27, 33, 39, 45),
                c(0, 2, 3, 5, 8, 9, 14, 15, 17, 20, 22, 25, 27, 30, 33, 46, 58))
liste4 <- lapply(liste_t, function(tk) {
  data.frame(t = tk, X = c(0, cumsum(rgamma(length(tk) - 1, 2 * diff(tk), 3))))
})

# Option 1: each trajectory up to its last inspection;
# step h = (8 + 12 + 45 + 58) / (2 + 4 + 10 + 16) = 3.84 for all
r5 <- gamma_bridge(liste4, a = 2, method = "MultiBridge")
r5
attr(r5, "teq")
sapply(r5, nrow)
plot_bridge(r5, trajectory = 1:4)

# The reconstructed increments of the four trajectories have the same law Gamma(a h, b):
# one sample for one test
d5 <- increments(r5)
ks.test(d5$dx, "pgamma", shape = 2 * attr(r5, "teq")[1], rate = 3)

# Option 2: all the trajectories up to 58, the shorter ones extended by gamma
# increments, which needs the rate b
r6 <- gamma_bridge(liste4, a = 2, method = "MultiBridge", option = 2, b = 3)
sapply(r6, nrow)
try(gamma_bridge(liste4, a = 2, method = "MultiBridge", option = 2))

# With one trajectory, "MultiBridge" is the method "Bridge"
attr(gamma_bridge(x, t, a = 2, method = "MultiBridge"), "teq")
```

```
# 10. Error: the first time must be 0
try(gamma_bridge(x, t + 1, a = 2))

# 11. Ten trajectories simulated with DegradSim, inspected more and more often
plan <- DegradSim::inspection_times("increasing", n = 20, n_traj = 10, tmax = 40)
sims <- DegradSim::sim_gamma(plan, a = 2, b = 3)

# Parameters estimated with levyfit
a_hat <- levyfit::gamma_mle(sims)[["a"]]
r <- gamma_bridge(sims, a = a_hat)
r
plot_bridge(r, trajectory = 1:4)

# Reconstructed increments of all the trajectories
d <- increments(r)
head(d)
# Under the model, the increments follow Gamma(a * teq, b)
ks.test(d$dx, "pgamma", shape = 2 * d$teq[1], rate = 3)
```

`ig_bridge` *Inverse Gaussian bridge: resampling on a regular grid*

Description

Resamples one or several trajectories of an inverse Gaussian process, observed at irregular inspection times, onto a regular time grid, by exact simulation of the inverse Gaussian bridge between the known values.

Usage

```
ig_bridge(
  x,
  t = NULL,
  eta,
  method = "Bridge",
  digits = NULL,
  common_step = FALSE,
  option = 1,
  mu = NULL
)
```

Arguments

`x`	Degradation levels of one trajectory; or a list of data frames, one per trajectory, whose first column contains the inspection times and second column the degradation levels (for instance the result of a `DegradSim` simulation function).
`t`	Inspection times of the trajectory, starting at 0 and strictly increasing. Not used when `x` is a list.
`eta`	Shape parameter $\eta > 0$, for instance the estimate `ig_mle(x)[["eta"]]` of the package `levyfit`.

`method`	`"Bridge"` (default) for the algorithm of Grall-Maës (2012), with as many grid points as observations and one grid per trajectory; `"ExtBridge"` for its extension, whose grid contains all the inspection times (see the section Method "ExtBridge"); `"MultiBridge"` for several trajectories on one common grid (see the section Method "MultiBridge").
`digits`	Used only with `method = "ExtBridge"`: number of decimal places of the inspection times, which sets the time step of the grid. `NULL` (default) finds it automatically.
`common_step`	Used only with `method = "ExtBridge"` and several trajectories: `FALSE` (default) gives each trajectory its own time step h; `TRUE` gives all the trajectories the same time step, the largest one such that every inspection time of every trajectory is a grid point. With a common step, the reconstructed increments of all the trajectories have the same law under the model and can be tested together.
`option`	Used only with `method = "MultiBridge"`: `1` (default) to resample each trajectory up to its own last inspection, `2` to resample all the trajectories up to the last inspection time over all the trajectories, extending the shorter ones by increments of the process (see the section Method "MultiBridge").
`mu`	Mean degradation per unit of time, $\mu > 0$; used only by the option 2 of the method "MultiBridge", to extend the trajectories.

Value

An object of class `"levybridge"`: a list with one data frame per trajectory, with columns `t` (regular grid) and `X` (resampled levels). Its attributes are `model`, `parameters`, `method`, `teq` (time step of the grid of each trajectory) and `observed` (the observed data, one data frame per trajectory; with the method "ExtBridge", the times are the rounded ones when a rounding was needed). Use `increments()` for the reconstructed increments and `plot_bridge()` to compare observed and resampled values.

Model

The inverse Gaussian process with parameters $\mu > 0$ and $\eta > 0$ has $X_0 = 0$, independent increments and $X_t - X_s \sim IG(\mu(t-s), \eta(t-s)^2)$ (mean $\mu(t-s)$, shape $\eta(t-s)^2$). Its trajectories are increasing.

Law of the bridge

Let $t_A < t < t_B$, $\tau_1 = t - t_A$, $\tau_2 = t_B - t$, $T = \tau_1 + \tau_2$, $\Delta = x_B - x_A > 0$, $m = \tau_1/\tau_2$ and $\lambda = \eta\tau_1^2/\Delta$. Given $X_{t_A} = x_A$ and $X_{t_B} = x_B$, the ratio $W = (X_t - x_A)/(x_B - X_t)$ ("realized / remaining") is the mixture

$$W \sim IG(m, \lambda) \text{ with probability } \tau_2/T, \qquad W \sim m^2/IG(m, \lambda) \text{ with probability } \tau_1/T,$$

and $X_t = x_A + \Delta\, W/(1+W)$. The law of the bridge does not depend on μ: only η is needed.

The simulation combines this mixture with the method of Michael, Schucany and Haas (1976), with one Gaussian and one uniform variable: with $q = Z^2$, $Z \sim \mathcal{N}(0,1)$, the two roots

$$w_2 = m + \frac{m^2 q}{2\lambda} + \frac{m}{2\lambda}\sqrt{4m\lambda q + m^2 q^2}, \qquad w_1 = m^2/w_2,$$

and $W = w_1$ with probability $\tau_1(1+w_1)/(T(m+w_1))$, $W = w_2$ otherwise. The small root is computed as m^2/w_2 to avoid a cancellation. When $\Delta = 0$ (two equal observed values, which can happen after rounding), the bridge is constant.

Resampling algorithm (method "Bridge")

This is the interpolation algorithm of Grall-Maës (2012), used for all the processes of the package. Let $0 = t_0 < t_1 < \cdots < t_N$ be the inspection times of a trajectory and $x_0, \ldots, x_N$ the observed levels. The regular grid is $t'_i = i\,\Delta t_{eq}$, $i = 0, \ldots, N$, with $\Delta t_{eq} = t_N/N$: it has as many points as observations and the same end points.

1. $x'_0 = x_0$ and $x'_N = x_N$.
2. For $i = 1, \ldots, N-1$, in increasing order:
   - if t'_i is an inspection time, x'_i is the observed value;
   - otherwise, x'_i is drawn from the bridge between the nearest known point on the left (an inspection or the grid point t'_{i-1} already drawn, whichever is closer) and the nearest inspection on the right.

By the Markov property of the bridge, the vector $(x'_1, \ldots, x'_{N-1})$ follows exactly the conditional law of $(X_{t'_1}, \ldots, X_{t'_{N-1}})$ given the observations. Hence, if the data come from the model and the parameters are the true ones, the reconstructed increments $x'_i - x'_{i-1}$ (see `increments()`) are independent with the law of $X_{\Delta t_{eq}}$: they can be tested with an i.i.d. goodness-of-fit test. A grid point that equals an inspection time up to rounding errors (relative difference below 10^{-9}) is set to that inspection time.

Precision. Each draw splits the increment Δ of the current interval into a left part (up to the grid point) and a right part (up to the next inspection), both computed directly: here ΔY and $\Delta(1-Y)$, with $Y = G_1/(G_1+G_2)$, $G_1 \sim \Gamma(a\tau_1)$, $G_2 \sim \Gamma(a\tau_2)$ drawn on the log scale. The reconstructed increments (`increments()`) are these parts, not differences of levels: a very small increment keeps all its significant digits instead of being rounded to 0, even for very small shapes $a\tau$; only an increment below the smallest positive number (about 10^{-308}), which needs extremely small shapes, is 0.

Method "ExtBridge"

Extension of the method of Grall-Maës whose regular grid contains all the inspection times (Tito and Paroissin). The inspection times are multiples of 10^{-d}, where d is the number of decimal places (argument `digits`), and the time step is the largest one such that every inspection time is a grid point:

$$h = 10^{-d}\,\gcd\!\left(10^d t_1, \ldots, 10^d t_N\right), \qquad t'_k = k\,h, \quad k = 0, \ldots, t_N/h.$$

The grid is then covered as in the method "Bridge": every inspection time keeps its observed value, and the other grid points are drawn from the bridge between the last known value on the left and the next inspection. The reconstructed increments are independent with the law of X_h, and the pseudo-sample contains all the observations. The grid has $t_N/h \geq N$ steps, with equality if and only if the inspection times are regular; then the method returns the observations themselves.

With `digits = NULL` (default), d is the smallest number of decimal places, between 0 and 6, such that all the inspection times are multiples of 10^{-d}. If there is none (for instance times drawn with `runif()`), the times are rounded to 2 decimal places, with a warning; give `digits` to choose another rounding. When `digits` is given and some times have more decimal places, they are rounded to `digits` decimal places, with a warning; two times equal after rounding give an error. A warning is also given when the grid has more than 10^5 steps, since the resampling then takes time. The grid and the time step are computed for each trajectory; with `common_step = TRUE`, d and h are computed from the inspection times of all the trajectories, so that all the grids have the same step.

Method "MultiBridge"

Version of the method "Bridge" for several trajectories, with **one regular grid common to all the trajectories** (Tito and Paroissin). With the method "Bridge", each trajectory has its own time step t_N/N: the reconstructed increments of two trajectories then have different laws and cannot be tested together. With the method "MultiBridge", all the reconstructed increments have the law of X_h for the same h: they form a single i.i.d. sample under the model, and one goodness-of-fit test can be applied to all the trajectories together.

Time step. For n trajectories, the trajectory i being observed on $[0, t_{i,max}]$ with m_i increments,

$$h = \frac{t_{1,max} + \cdots + t_{n,max}}{m_1 + \cdots + m_n},$$

the total observed duration divided by the total number of increments. The number of reconstructed increments is then about the number of observed increments, as with the method "Bridge"; for one trajectory, $h = t_N/N$ is the step of the method "Bridge". The grid is $t'_k = kh$; as with the method "Bridge", it does not necessarily contain the inspection times (a grid point equal to an inspection time keeps the observed value).

Option 1 (`option = 1`, default): each trajectory is resampled up to its own last inspection, on the grid points $kh \leq t_{i,max}$. Each grid point is drawn from the bridge between the last known value on the left and the next inspection, exactly as with the method "Bridge". The end of the trajectory, between its last grid point and $t_{i,max}$, is not used. A long trajectory gives many reconstructed increments, a short one few (none when $t_{i,max} < h$, with a warning). Under the model, with the true parameters, all the reconstructed increments are independent with the law of X_h: the pooled sample is exactly i.i.d. This is the recommended option.

Option 2 (`option = 2`): all the trajectories are resampled up to the last inspection time over all the trajectories, $\max_i t_{i,max}$, so that they all have the same number of grid points. Up to $t_{i,max}$, as in option 1; beyond, there is no observation left and the trajectory is extended by increments of the process drawn with the given parameters (not a bridge): the first one starts from the last observation $x_{i,N}$. This needs all the parameters of the process (for instance b for the gamma process, see the arguments). Under the model, with the true parameters, the pooled sample is also i.i.d. with the law of X_h; but the extended values do not come from the data: they follow the null model by construction and dilute the information of the observations, which lowers the power of a test, all the more as the trajectories have different durations. Use it only for trajectories of similar durations.

When to use "MultiBridge". For several trajectories observed at different times, when one test on all the trajectories together is wanted. With one trajectory, both options give the method "Bridge". The method "ExtBridge" with `common_step = TRUE` also gives a common step, with a grid that contains all the inspection times, but it can have many more grid points.

References

Grall-Maës, E. (2012). Use of the Kolmogorov-Smirnov test for gamma process. *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability*, 226(6), 624-634. doi:10.1177/1748006X12462522

Michael, J. R., Schucany, W. R. and Haas, R. W. (1976). Generating random variates using transformations with multiple roots. *The American Statistician*, 30(2), 88-90. doi:10.1080/00031305.1976.10479147

Ribeiro, C. and Webber, N. (2003). A Monte Carlo method for the normal inverse Gaussian option valuation model using an inverse Gaussian bridge. *Computing in Economics and Finance 2003*, 5.

See Also

- `increments()`: the reconstructed increments;

- plot_bridge(): observed and resampled values;
- gamma_bridge(): bridge of the gamma process;
- wiener_bridge(): bridge of the Wiener process;
- vg_bridge(): bridge of the variance-gamma process;
- meixner_bridge(): bridge of the Meixner process.

Examples

```
# 1. One trajectory observed at 11 irregular times (mu = 1, eta = 3)
t <- c(0, 1.2, 2, 3.8, 5.5, 7.1, 10.4, 12.9, 15.3, 18.7, 22.5)
x <- c(0, 1.31, 2.05, 3.62, 5.71, 7.15, 10.62, 12.83, 15.02, 18.94, 22.31)
set.seed(1)
r <- ig_bridge(x, t, eta = 3)
r

# The resampled trajectory, on the regular grid 0, 2.25, 4.5, ..., 22.5
r[[1]]

# Time step of the grid: t_N / N = 22.5 / 10
attr(r, "teq")

# 2. Reconstructed increments, the input of the goodness-of-fit tests
increments(r)

# 3. Observed and resampled values
plot_bridge(r)

# 4. Grid points equal to inspection times keep the observed values:
#    here the grid is 0, 2, 4, 6, 8, 10 and 2 and 6 are inspection times
t2 <- c(0, 2, 3, 6, 7, 10)
x2 <- c(0, 1.4, 2.0, 4.1, 4.6, 6.8)
ig_bridge(x2, t2, eta = 3)[[1]]

# 5. One trajectory given as a data frame (times, then levels)
ig_bridge(data.frame(t = t2, X = x2), eta = 3)

# 6. Several trajectories, each with its own inspection times:
#    one regular grid and one time step per trajectory
liste <- list(data.frame(t = c(0, 1, 3, 6), X = c(0, 0.8, 2.1, 4.3)),
              data.frame(t = c(0, 2, 4, 5, 9), X = c(0, 1.2, 2.9, 3.1, 6.2)))
r2 <- ig_bridge(liste, eta = 3)
r2
attr(r2, "teq")
increments(r2)
plot_bridge(r2, trajectory = 1:2)

# 7. The bridge values are random: set.seed() makes them reproducible
set.seed(2); y1 <- ig_bridge(x, t, eta = 3)[[1]]$X
set.seed(2); y2 <- ig_bridge(x, t, eta = 3)[[1]]$X
identical(y1, y2)

# 8. Equal observed values (after rounding) give a constant bridge,
#    a decrease is impossible for this increasing process
ig_bridge(c(0, 1, 1, 2), c(0, 1, 2.5, 3), eta = 3)[[1]]
try(ig_bridge(c(0, 2, 1.5, 3), c(0, 1, 2, 3), eta = 3))
```

```
# 9. Method "ExtBridge": the times have 1 decimal place, the grid
#    0, 0.1, ..., 22.5 contains all of them
r3 <- ig_bridge(x, t, eta = 3, method = "ExtBridge")
attr(r3, "teq")
all(t %in% r3[[1]]$t)
plot_bridge(r3)

# Error: the first time must be 0
try(ig_bridge(x, t + 1, eta = 3))

# Method "MultiBridge": several trajectories with their own times on one common grid,
# so that all the reconstructed increments have the same law
liste4 <- list(data.frame(t = c(0, 3, 8), X = c(0, 1.5, 4.2)),
               data.frame(t = c(0, 2, 5, 9, 12), X = c(0, 0.9, 2.8, 4.1, 6.5)),
               data.frame(t = c(0, 4, 7, 11, 15), X = c(0, 2.2, 3.1, 5.6, 7.4)))
r4 <- ig_bridge(liste4, eta = 3, method = "MultiBridge")
attr(r4, "teq")
sapply(r4, nrow)

# 10. Ten trajectories simulated with DegradSim, inspected more and more often
plan <- DegradSim::inspection_times("increasing", n = 20, n_traj = 10, tmax = 40)
sims <- DegradSim::sim_ig(plan, mu = 1, eta = 3)

# Parameters estimated with levyfit
eta_hat <- levyfit::ig_mle(sims)[["eta"]]
r <- ig_bridge(sims, eta = eta_hat)
r
plot_bridge(r, trajectory = 1:4)

# Reconstructed increments of all the trajectories
d <- increments(r)
head(d)
# Under the model, the increments follow IG(mu * teq, eta * teq^2)
# (mean mu * teq, variance mu^3 * teq / eta)
c(mean(d$dx), 1 * d$teq[1], var(d$dx), d$teq[1] / 3)
```

`increments` *Reconstructed increments*

Description

Extracts the increments of the trajectories resampled on the regular grid by a `_bridge` function. Under the model, with the true parameters, these increments are independent and have the law of an increment of the process over a time step `teq` (Grall-Maës, 2012): they are the input of the goodness-of-fit tests.

Usage

```
increments(x)
```

Arguments

`x`	Result of a bridge function of the package, for instance `gamma_bridge()`.

Value

A data frame with one row per increment and three columns:

- `id`: number of the trajectory (1, 2, ...);
- `teq`: time step of the regular grid of this trajectory, t_N/N for the method "Bridge" and h for the method "ExtBridge";
- `dx`: reconstructed increment $x'_i - x'_{i-1}$.

The increments are computed directly by the bridges, not as differences of the resampled levels: a very small increment keeps its relative precision instead of being rounded to 0 (the levels have about 16 significant digits, so an increment smaller than about 10^{-16} times the level would be lost by a difference).

References

Grall-Maës, E. (2012). Use of the Kolmogorov-Smirnov test for gamma process. *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability*, 226(6), 624-634. doi:10.1177/1748006X12462522

Tito, A. R. and Paroissin, C. (in preparation). (methods "ExtBridge" and "MultiBridge")

See Also

`gamma_bridge()`, `wiener_bridge()`, `ig_bridge()`, `vg_bridge()`, `meixner_bridge()`

Examples

```
# One gamma trajectory observed at irregular times
t <- c(0, 1.2, 2, 3.8, 5.5, 7.1, 10.4, 12.9, 15.3, 18.7, 22.5)
set.seed(1)
x <- c(0, cumsum(rgamma(10, shape = 2 * diff(t), rate = 3)))
r <- gamma_bridge(x, t, a = 2)
increments(r)

# Two trajectories with different times: one time step per trajectory
liste <- list(data.frame(t = c(0, 1, 3, 6), X = c(0, 0.8, 2.1, 4.3)),
              data.frame(t = c(0, 2, 4, 5, 9), X = c(0, 1.2, 2.9, 3.1, 6.2)))
increments(gamma_bridge(liste, a = 2))

# Method "MultiBridge": one common grid, so all the increments have the same
# law and form one sample
d <- increments(gamma_bridge(liste, a = 2, method = "MultiBridge"))
d
unique(d$teq)

# The increments are computed directly: very small ones are not rounded to 0
min(increments(gamma_bridge(x, t, a = 0.5, method = "ExtBridge"))$dx)
```

meixner_bridge *Meixner bridge: resampling on a regular grid*

Description

Resamples one or several trajectories of a Meixner process, observed at irregular inspection times, onto a regular time grid, by exact simulation of the Meixner bridge between the known values.

Usage

```
meixner_bridge(
  x,
  t = NULL,
  a,
  d,
  m,
  method = "Bridge",
  digits = NULL,
  common_step = FALSE,
  option = 1,
  b = NULL
)
```

Arguments

x	Degradation levels of one trajectory; or a list of data frames, one per trajectory, whose first column contains the inspection times and second column the degradation levels (for instance the result of a DegradSim simulation function).
t	Inspection times of the trajectory, starting at 0 and strictly increasing. Not used when x is a list.
a	Scale parameter, $a > 0$.
d	Shape parameter per unit of time, $d > 0$.
m	Location parameter per unit of time, real.
method	"Bridge" (default) for the algorithm of Grall-Maës (2012), with as many grid points as observations and one grid per trajectory; "ExtBridge" for its extension, whose grid contains all the inspection times (see the section Method "ExtBridge"); "MultiBridge" for several trajectories on one common grid (see the section Method "MultiBridge").
digits	Used only with method = "ExtBridge": number of decimal places of the inspection times, which sets the time step of the grid. NULL (default) finds it automatically.
common_step	Used only with method = "ExtBridge" and several trajectories: FALSE (default) gives each trajectory its own time step h; TRUE gives all the trajectories the same time step, the largest one such that every inspection time of every trajectory is a grid point. With a common step, the reconstructed increments of all the trajectories have the same law under the model and can be tested together.
option	Used only with method = "MultiBridge": 1 (default) to resample each trajectory up to its own last inspection, 2 to resample all the trajectories up to the last inspection time over all the trajectories, extending the shorter ones by increments of the process (see the section Method "MultiBridge").

b — Skewness parameter, $b \in]-\pi, \pi[$; used only by the option 2 of the method "MultiBridge", to extend the trajectories.

Value

An object of class `"levybridge"`: a list with one data frame per trajectory, with columns `t` (regular grid) and `X` (resampled levels). Its attributes are `model`, `parameters`, `method`, `teq` (time step of the grid of each trajectory) and `observed` (the observed data, one data frame per trajectory; with the method "ExtBridge", the times are the rounded ones when a rounding was needed). Use `increments()` for the reconstructed increments and `plot_bridge()` to compare observed and resampled values.

Model

The $Meixner(a, b, d, m)$ distribution, $a > 0$, $b \in]-\pi, \pi[$, $d > 0$, m real, has density

$$f(x) = \frac{(2\cos(b/2))^{2d}}{2a\pi\,\Gamma(2d)} \exp\left(\frac{b(x-m)}{a}\right) \left|\Gamma\left(d + i\frac{x-m}{a}\right)\right|^2.$$

The Meixner process has $X_0 = 0$, independent increments and $X_t - X_s \sim Meixner(a, b, d(t-s), m(t-s))$. Its trajectories are not monotone.

Law of the bridge

Let $t_A < t < t_B$, $h_\alpha(y) = |\Gamma(\alpha + iy)|^2$,

$$\alpha_1 = d(t - t_A), \qquad \alpha_2 = d(t_B - t), \qquad z = \frac{(x_B - x_A) - m(t_B - t_A)}{a}.$$

Given $X_{t_A} = x_A$ and $X_{t_B} = x_B$, $X_t = x_A + m(t - t_A) + aY$, where Y has the density

$$q(y) = \frac{h_{\alpha_1}(y)\, h_{\alpha_2}(z - y)}{2\pi B(2\alpha_1, 2\alpha_2)\, h_{\alpha_1+\alpha_2}(z)}.$$

The law of the bridge does not depend on b: only a, d and m are needed.

Simulation of the bridge

Y is drawn exactly by acceptance-rejection with a Cauchy envelope (algorithm 6 of Tito and Paroissin). For $\alpha > 0$ and y real, $h_\alpha(y) \le \Gamma(\alpha)^2$ and $h_\alpha(y) \le \Gamma(\alpha)^2\alpha^2/(\alpha^2 + y^2)$. With $(y_0, s) = (0, \alpha_1)$ if $\alpha_1 \le \alpha_2$ and $(y_0, s) = (z, \alpha_2)$ otherwise, this gives, for all y,

$$h_{\alpha_1}(y)\, h_{\alpha_2}(z - y) \le \Gamma(\alpha_1)^2\Gamma(\alpha_2)^2 \frac{s^2}{s^2 + (y - y_0)^2},$$

a multiple of the Cauchy density of centre y_0 and scale s. The algorithm repeats, with U and V independent uniform variables on $(0, 1)$:

1. $Y = y_0 + s\tan(\pi(V - 1/2))$, a draw of the Cauchy law;
2. accept Y if
$$U\,\Gamma(\alpha_1)^2\Gamma(\alpha_2)^2 \frac{s^2}{s^2 + (Y - y_0)^2} \le h_{\alpha_1}(Y)\, h_{\alpha_2}(z - Y),$$
otherwise draw again.

The number of draws follows a geometric law with parameter

$$P_{acc} = \frac{2B(2\alpha_1, 2\alpha_2)\, h_{\alpha_1+\alpha_2}(z)}{s\,\Gamma(\alpha_1)^2\Gamma(\alpha_2)^2}.$$

P_{acc} decreases quickly when $|z|$ increases (for $\alpha_1 = \alpha_2 = 1$: 1/3 if $z = 0$, 0.04 if $z = 2$, $5 \cdot 10^{-4}$ if $z = 4$): an increment far from its mean $m(t_B - t_A)$ can need many draws, as well as large values of α_1 and α_2 (down to $P_{acc} \approx 10^{-8}$ for parameters estimated on ordinary trajectories).

Since P_{acc} is known exactly, the algorithm is chosen before drawing: the Cauchy envelope when $P_{acc} \geq 0.05$, otherwise an envelope constant on cells of width 0.1 covering the region between 0 and z (refined near the peaks at 0 and z when α is small), with Cauchy tails outside; it is valid because $h_\alpha(y)$ decreases with $|y|$, and its acceptance rate stays high for all the parameters (about 0.6 ms per draw in the cases above). The choice depends only on (α_1, α_2, z), not on the draws: Y keeps exactly the law q. The function $\log h_\alpha$ is computed with `gsl::lngamma_complex()`.

Resampling algorithm (method "Bridge")

This is the interpolation algorithm of Grall-Maës (2012), used for all the processes of the package. Let $0 = t_0 < t_1 < \cdots < t_N$ be the inspection times of a trajectory and $x_0, \ldots, x_N$ the observed levels. The regular grid is $t'_i = i\,\Delta t_{eq}$, $i = 0, \ldots, N$, with $\Delta t_{eq} = t_N/N$: it has as many points as observations and the same end points.

1. $x'_0 = x_0$ and $x'_N = x_N$.
2. For $i = 1, \ldots, N-1$, in increasing order:
   - if t'_i is an inspection time, x'_i is the observed value;
   - otherwise, x'_i is drawn from the bridge between the nearest known point on the left (an inspection or the grid point t'_{i-1} already drawn, whichever is closer) and the nearest inspection on the right.

By the Markov property of the bridge, the vector $(x'_1, \ldots, x'_{N-1})$ follows exactly the conditional law of $(X_{t'_1}, \ldots, X_{t'_{N-1}})$ given the observations. Hence, if the data come from the model and the parameters are the true ones, the reconstructed increments $x'_i - x'_{i-1}$ (see `increments()`) are independent with the law of $X_{\Delta t_{eq}}$: they can be tested with an i.i.d. goodness-of-fit test. A grid point that equals an inspection time up to rounding errors (relative difference below 10^{-9}) is set to that inspection time.

Precision. Each draw splits the increment Δ of the current interval into a left part (up to the grid point) and a right part (up to the next inspection), both computed directly: here ΔY and $\Delta(1-Y)$, with $Y = G_1/(G_1+G_2)$, $G_1 \sim \Gamma(a\tau_1)$, $G_2 \sim \Gamma(a\tau_2)$ drawn on the log scale. The reconstructed increments (`increments()`) are these parts, not differences of levels: a very small increment keeps all its significant digits instead of being rounded to 0, even for very small shapes $a\tau$; only an increment below the smallest positive number (about 10^{-308}), which needs extremely small shapes, is 0.

Method "ExtBridge"

Extension of the method of Grall-Maës whose regular grid contains all the inspection times (Tito and Paroissin). The inspection times are multiples of 10^{-d}, where d is the number of decimal places (argument `digits`), and the time step is the largest one such that every inspection time is a grid point:

$$h = 10^{-d}\,\gcd\big(10^d t_1, \ldots, 10^d t_N\big), \qquad t'_k = k\,h, \quad k = 0, \ldots, t_N/h.$$

The grid is then covered as in the method "Bridge": every inspection time keeps its observed value, and the other grid points are drawn from the bridge between the last known value on the left and

the next inspection. The reconstructed increments are independent with the law of X_h, and the pseudo-sample contains all the observations. The grid has $t_N/h \geq N$ steps, with equality if and only if the inspection times are regular; then the method returns the observations themselves.

With `digits = NULL` (default), d is the smallest number of decimal places, between 0 and 6, such that all the inspection times are multiples of 10^{-d}. If there is none (for instance times drawn with `runif()`), the times are rounded to 2 decimal places, with a warning; give `digits` to choose another rounding. When `digits` is given and some times have more decimal places, they are rounded to `digits` decimal places, with a warning; two times equal after rounding give an error. A warning is also given when the grid has more than 10^5 steps, since the resampling then takes time. The grid and the time step are computed for each trajectory; with `common_step = TRUE`, d and h are computed from the inspection times of all the trajectories, so that all the grids have the same step.

Method "MultiBridge"

Version of the method "Bridge" for several trajectories, with **one regular grid common to all the trajectories** (Tito and Paroissin). With the method "Bridge", each trajectory has its own time step t_N/N: the reconstructed increments of two trajectories then have different laws and cannot be tested together. With the method "MultiBridge", all the reconstructed increments have the law of X_h for the same h: they form a single i.i.d. sample under the model, and one goodness-of-fit test can be applied to all the trajectories together.

Time step. For n trajectories, the trajectory i being observed on $[0, t_{i,max}]$ with m_i increments,

$$h = \frac{t_{1,max} + \cdots + t_{n,max}}{m_1 + \cdots + m_n},$$

the total observed duration divided by the total number of increments. The number of reconstructed increments is then about the number of observed increments, as with the method "Bridge"; for one trajectory, $h = t_N/N$ is the step of the method "Bridge". The grid is $t'_k = kh$; as with the method "Bridge", it does not necessarily contain the inspection times (a grid point equal to an inspection time keeps the observed value).

Option 1 (`option = 1`, default): each trajectory is resampled up to its own last inspection, on the grid points $kh \leq t_{i,max}$. Each grid point is drawn from the bridge between the last known value on the left and the next inspection, exactly as with the method "Bridge". The end of the trajectory, between its last grid point and $t_{i,max}$, is not used. A long trajectory gives many reconstructed increments, a short one few (none when $t_{i,max} < h$, with a warning). Under the model, with the true parameters, all the reconstructed increments are independent with the law of X_h: the pooled sample is exactly i.i.d. This is the recommended option.

Option 2 (`option = 2`): all the trajectories are resampled up to the last inspection time over all the trajectories, $\max_i t_{i,max}$, so that they all have the same number of grid points. Up to $t_{i,max}$, as in option 1; beyond, there is no observation left and the trajectory is extended by increments of the process drawn with the given parameters (not a bridge): the first one starts from the last observation $x_{i,N}$. This needs all the parameters of the process (for instance b for the gamma process, see the arguments). Under the model, with the true parameters, the pooled sample is also i.i.d. with the law of X_h; but the extended values do not come from the data: they follow the null model by construction and dilute the information of the observations, which lowers the power of a test, all the more as the trajectories have different durations. Use it only for trajectories of similar durations.

When to use "MultiBridge". For several trajectories observed at different times, when one test on all the trajectories together is wanted. With one trajectory, both options give the method "Bridge". The method "ExtBridge" with `common_step = TRUE` also gives a common step, with a grid that contains all the inspection times, but it can have many more grid points.

References

Grall-Maës, E. (2012). Use of the Kolmogorov-Smirnov test for gamma process. *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability*, 226(6), 624-634. doi:10.1177/1748006X12462522

Schoutens, W. and Teugels, J. L. (1998). Lévy processes, polynomials and martingales. *Communications in Statistics. Stochastic Models*, 14(1-2), 335-349.

Devroye, L. (1986). *Non-Uniform Random Variate Generation*. Springer.

Tito, A. R. and Paroissin, C. (in preparation).

See Also

- `increments()`: the reconstructed increments;
- `plot_bridge()`: observed and resampled values;
- `gamma_bridge()`: bridge of the gamma process;
- `wiener_bridge()`: bridge of the Wiener process;
- `ig_bridge()`: bridge of the inverse Gaussian process;
- `vg_bridge()`: bridge of the variance-gamma process.

Examples

```
# 1. One trajectory observed at 11 irregular times (a = 1, b = 0.3, d = 1.5, m = 0.2)
t <- c(0, 1.2, 2, 3.8, 5.5, 7.1, 10.4, 12.9, 15.3, 18.7, 22.5)
x <- c(0, 0.9, 0.4, 2.8, 3.1, 5.6, 7.2, 6.5, 9.8, 11.3, 12.1)
set.seed(1)
r <- meixner_bridge(x, t, a = 1, d = 1.5, m = 0.2)
r

# The resampled trajectory, on the regular grid 0, 2.25, 4.5, ..., 22.5
r[[1]]

# Time step of the grid: t_N / N = 22.5 / 10
attr(r, "teq")

# 2. Reconstructed increments, the input of the goodness-of-fit tests
increments(r)

# 3. Observed and resampled values
plot_bridge(r)

# 4. Grid points equal to inspection times keep the observed values:
#    here the grid is 0, 2, 4, 6, 8, 10 and 2 and 6 are inspection times
t2 <- c(0, 2, 3, 6, 7, 10)
x2 <- c(0, 1.4, 0.9, 3.1, 2.6, 4.8)
meixner_bridge(x2, t2, a = 1, d = 1.5, m = 0.2)[[1]]

# 5. One trajectory given as a data frame (times, then levels)
meixner_bridge(data.frame(t = t2, X = x2), a = 1, d = 1.5, m = 0.2)

# 6. Several trajectories, each with its own inspection times:
#    one regular grid and one time step per trajectory
liste <- list(data.frame(t = c(0, 1, 3, 6), X = c(0, 0.4, 2.3, 1.9)),
              data.frame(t = c(0, 2, 4, 5, 9), X = c(0, 1.6, 0.8, 3.1, 4.2)))
r2 <- meixner_bridge(liste, a = 1, d = 1.5, m = 0.2)
```

```
r2
attr(r2, "teq")
increments(r2)
plot_bridge(r2, trajectory = 1:2)

# 7. The bridge values are random: set.seed() makes them reproducible
set.seed(2); y1 <- meixner_bridge(x, t, a = 1, d = 1.5, m = 0.2)[[1]]$X
set.seed(2); y2 <- meixner_bridge(x, t, a = 1, d = 1.5, m = 0.2)[[1]]$X
identical(y1, y2)

# 8. Method "ExtBridge": the times have 1 decimal place, the grid
#    0, 0.1, ..., 22.5 contains all of them
r3 <- meixner_bridge(x, t, a = 1, d = 1.5, m = 0.2, method = "ExtBridge")
attr(r3, "teq")
all(t %in% r3[[1]]$t)
plot_bridge(r3)

# Error: the first time must be 0
try(meixner_bridge(x, t + 1, a = 1, d = 1.5, m = 0.2))

# Method "MultiBridge": several trajectories with their own times on one common grid,
# so that all the reconstructed increments have the same law
liste4 <- list(data.frame(t = c(0, 3, 8), X = c(0, 1.5, 4.2)),
               data.frame(t = c(0, 2, 5, 9, 12), X = c(0, 0.9, 2.8, 4.1, 6.5)),
               data.frame(t = c(0, 4, 7, 11, 15), X = c(0, 2.2, 3.1, 5.6, 7.4)))
r4 <- meixner_bridge(liste4, a = 1, d = 1.5, m = 0.2, method = "MultiBridge")
attr(r4, "teq")
sapply(r4, nrow)

# 9. Ten trajectories simulated with DegradSim, inspected more and more often
plan <- DegradSim::inspection_times("increasing", n = 20, n_traj = 10, tmax = 40)
sims <- DegradSim::sim_meixner(plan, a = 1, b = 0.3, d = 1.5, m = 0.2)

# With the true parameters (b is not needed by the bridge)
r <- meixner_bridge(sims, a = 1, d = 1.5, m = 0.2)
r
plot_bridge(r, trajectory = 1:4)

# Reconstructed increments of all the trajectories
d <- increments(r)
head(d)
# Under the model, the increments have mean (m + a d tan(b/2)) teq
# and variance a^2 d teq / (2 cos(b/2)^2)
c(mean(d$dx), (0.2 + 1.5 * tan(0.15)) * d$teq[1])
c(var(d$dx), 1.5 * d$teq[1] / (2 * cos(0.15)^2))
```

`plot_bridge` *Plot observed and resampled trajectories*

Description

Draws, for one or several trajectories, the observed values at the inspection times and the values resampled on the regular grid by a `_bridge` function, with `ggplot2`. The plot shows how the bridge fills the gaps between the inspections.

Usage

```
plot_bridge(x, trajectory = 1, title = NULL)
```

Arguments

x	Result of a bridge function of the package, for instance gamma_bridge().
trajectory	Numbers of the trajectories to plot, for instance 1 or 1:3 (default: the first one). With several trajectories, one panel is drawn per trajectory.
title	Title of the plot, a character string; by default the name of the bridge, for instance "gamma bridge".

Value

A ggplot object. It is displayed when printed, and it can be modified with the usual ggplot2 functions, added with +: for instance + ggplot2::labs(x = "Time (hours)").

See Also

increments()

Examples

```
t <- c(0, 1.2, 2, 3.8, 5.5, 7.1, 10.4, 12.9, 15.3, 18.7, 22.5)
set.seed(1)
x <- c(0, cumsum(rgamma(10, shape = 2 * diff(t), rate = 3)))
r <- gamma_bridge(x, t, a = 2)

# Observed values (red) and resampled values on the regular grid (blue)
plot_bridge(r)

# Custom title and axis labels
plot_bridge(r, title = "Gamma bridge, a = 2") +
  ggplot2::labs(x = "Time (hours)", y = "Degradation (%)")

# Several trajectories: one panel per trajectory
liste <- list(data.frame(t = c(0, 1, 3, 6), X = c(0, 0.8, 2.1, 4.3)),
              data.frame(t = c(0, 2, 4, 5, 9), X = c(0, 1.2, 2.9, 3.1, 6.2)))
plot_bridge(gamma_bridge(liste, a = 2), trajectory = 1:2)
```

---

vg_bridge *Variance-gamma bridge: resampling on a regular grid*

Description

Resamples one or several trajectories of a variance-gamma process, observed at irregular inspection times, onto a regular time grid, by exact simulation of the variance-gamma bridge between the known values.

Usage

```
vg_bridge(
  x,
  t = NULL,
  theta,
  sigma,
  nu,
  method = "Bridge",
  digits = NULL,
  common_step = FALSE,
  option = 1
)
```

Arguments

x	Degradation levels of one trajectory; or a list of data frames, one per trajectory, whose first column contains the inspection times and second column the degradation levels (for instance the result of a DegradSim simulation function).
t	Inspection times of the trajectory, starting at 0 and strictly increasing. Not used when x is a list.
theta	Drift of the subordinated Brownian motion, real.
sigma	Volatility of the subordinated Brownian motion, $\sigma > 0$.
nu	Variance rate of the gamma clock, $\nu > 0$.
method	"Bridge" (default) for the algorithm of Grall-Maës (2012), with as many grid points as observations and one grid per trajectory; "ExtBridge" for its extension, whose grid contains all the inspection times (see the section Method "ExtBridge"); "MultiBridge" for several trajectories on one common grid (see the section Method "MultiBridge").
digits	Used only with method = "ExtBridge": number of decimal places of the inspection times, which sets the time step of the grid. NULL (default) finds it automatically.
common_step	Used only with method = "ExtBridge" and several trajectories: FALSE (default) gives each trajectory its own time step h; TRUE gives all the trajectories the same time step, the largest one such that every inspection time of every trajectory is a grid point. With a common step, the reconstructed increments of all the trajectories have the same law under the model and can be tested together.
option	Used only with method = "MultiBridge": 1 (default) to resample each trajectory up to its own last inspection, 2 to resample all the trajectories up to the last inspection time over all the trajectories, extending the shorter ones by increments of the process (see the section Method "MultiBridge").

Value

An object of class "levybridge": a list with one data frame per trajectory, with columns t (regular grid) and X (resampled levels). Its attributes are model, parameters, method, teq (time step of the grid of each trajectory) and observed (the observed data, one data frame per trajectory; with the method "ExtBridge", the times are the rounded ones when a rounding was needed). Use increments() for the reconstructed increments and plot_bridge() to compare observed and resampled values.

Model

The variance-gamma process with parameters θ, $\sigma > 0$ and $\nu > 0$ is $X_s = \theta\gamma_s + \sigma W_{\gamma_s}$, where γ is a gamma process with $\gamma_s \sim \Gamma(s/\nu)$ with scale ν (mean s, variance νs) and W an independent standard Brownian motion. Its trajectories are not monotone.

Law of the bridge

Let $t_A < t < t_B$, $\tau_1 = t - t_A$, $\tau_2 = t_B - t$, $T = \tau_1 + \tau_2$, $\Delta = x_B - x_A$,

$$p = \frac{T}{\nu} - \frac{1}{2}, \qquad \kappa^2 = \frac{\theta^2 + 2\sigma^2/\nu}{\sigma^4}.$$

Given $X_{t_A} = x_A$ and $X_{t_B} = x_B$,

$$X_t = x_A + Y\Delta + \sqrt{S\,Y(1-Y)}\,\xi,$$

with three independent variables $S \sim \mathrm{GIG}(p, \kappa^2, \Delta^2)$ (density proportional to $s^{p-1}\exp(-(\kappa^2 s + \Delta^2/s)/2)$), $Y \sim \mathrm{Beta}(\tau_1/\nu, \tau_2/\nu)$ and $\xi \sim \mathcal{N}(0,1)$. S/σ^2 is the increment of the clock γ over $[t_A, t_B]$, Y the fraction of it reached at t, and, given the clock, X is a Brownian bridge. The law of the bridge depends on (θ, σ, ν) only through ν and κ. The GIG variable is drawn with `GIGrvg::rgig()`. The case $\Delta = 0$ with $p \le 0$, which has probability zero, gives $S = 0$.

Resampling algorithm (method "Bridge")

This is the interpolation algorithm of Grall-Maës (2012), used for all the processes of the package. Let $0 = t_0 < t_1 < \cdots < t_N$ be the inspection times of a trajectory and $x_0, \ldots, x_N$ the observed levels. The regular grid is $t'_i = i\,\Delta t_{eq}$, $i = 0, \ldots, N$, with $\Delta t_{eq} = t_N/N$: it has as many points as observations and the same end points.

1. $x'_0 = x_0$ and $x'_N = x_N$.
2. For $i = 1, \ldots, N-1$, in increasing order:
   - if t'_i is an inspection time, x'_i is the observed value;
   - otherwise, x'_i is drawn from the bridge between the nearest known point on the left (an inspection or the grid point t'_{i-1} already drawn, whichever is closer) and the nearest inspection on the right.

By the Markov property of the bridge, the vector $(x'_1, \ldots, x'_{N-1})$ follows exactly the conditional law of $(X_{t'_1}, \ldots, X_{t'_{N-1}})$ given the observations. Hence, if the data come from the model and the parameters are the true ones, the reconstructed increments $x'_i - x'_{i-1}$ (see `increments()`) are independent with the law of $X_{\Delta t_{eq}}$: they can be tested with an i.i.d. goodness-of-fit test. A grid point that equals an inspection time up to rounding errors (relative difference below 10^{-9}) is set to that inspection time.

Precision. Each draw splits the increment Δ of the current interval into a left part (up to the grid point) and a right part (up to the next inspection), both computed directly: here $\Delta\,Y$ and $\Delta(1-Y)$, with $Y = G_1/(G_1+G_2)$, $G_1 \sim \Gamma(a\tau_1)$, $G_2 \sim \Gamma(a\tau_2)$ drawn on the log scale. The reconstructed increments (`increments()`) are these parts, not differences of levels: a very small increment keeps all its significant digits instead of being rounded to 0, even for very small shapes $a\tau$; only an increment below the smallest positive number (about 10^{-308}), which needs extremely small shapes, is 0.

Method "ExtBridge"

Extension of the method of Grall-Maës whose regular grid contains all the inspection times (Tito and Paroissin). The inspection times are multiples of 10^{-d}, where d is the number of decimal places (argument `digits`), and the time step is the largest one such that every inspection time is a grid point:

$$h = 10^{-d}\,\gcd\bigl(10^d t_1, \ldots, 10^d t_N\bigr), \qquad t'_k = k\,h, \quad k = 0, \ldots, t_N/h.$$

The grid is then covered as in the method "Bridge": every inspection time keeps its observed value, and the other grid points are drawn from the bridge between the last known value on the left and the next inspection. The reconstructed increments are independent with the law of X_h, and the pseudo-sample contains all the observations. The grid has $t_N/h \geq N$ steps, with equality if and only if the inspection times are regular; then the method returns the observations themselves.

With `digits = NULL` (default), d is the smallest number of decimal places, between 0 and 6, such that all the inspection times are multiples of 10^{-d}. If there is none (for instance times drawn with `runif()`), the times are rounded to 2 decimal places, with a warning; give `digits` to choose another rounding. When `digits` is given and some times have more decimal places, they are rounded to `digits` decimal places, with a warning; two times equal after rounding give an error. A warning is also given when the grid has more than 10^5 steps, since the resampling then takes time. The grid and the time step are computed for each trajectory; with `common_step = TRUE`, d and h are computed from the inspection times of all the trajectories, so that all the grids have the same step.

Method "MultiBridge"

Version of the method "Bridge" for several trajectories, with **one regular grid common to all the trajectories** (Tito and Paroissin). With the method "Bridge", each trajectory has its own time step t_N/N: the reconstructed increments of two trajectories then have different laws and cannot be tested together. With the method "MultiBridge", all the reconstructed increments have the law of X_h for the same h: they form a single i.i.d. sample under the model, and one goodness-of-fit test can be applied to all the trajectories together.

Time step. For n trajectories, the trajectory i being observed on $[0, t_{i,max}]$ with m_i increments,

$$h = \frac{t_{1,max} + \cdots + t_{n,max}}{m_1 + \cdots + m_n},$$

the total observed duration divided by the total number of increments. The number of reconstructed increments is then about the number of observed increments, as with the method "Bridge"; for one trajectory, $h = t_N/N$ is the step of the method "Bridge". The grid is $t'_k = kh$; as with the method "Bridge", it does not necessarily contain the inspection times (a grid point equal to an inspection time keeps the observed value).

Option 1 (`option = 1`, default): each trajectory is resampled up to its own last inspection, on the grid points $kh \leq t_{i,max}$. Each grid point is drawn from the bridge between the last known value on the left and the next inspection, exactly as with the method "Bridge". The end of the trajectory, between its last grid point and $t_{i,max}$, is not used. A long trajectory gives many reconstructed increments, a short one few (none when $t_{i,max} < h$, with a warning). Under the model, with the true parameters, all the reconstructed increments are independent with the law of X_h: the pooled sample is exactly i.i.d. This is the recommended option.

Option 2 (`option = 2`): all the trajectories are resampled up to the last inspection time over all the trajectories, $\max_i t_{i,max}$, so that they all have the same number of grid points. Up to $t_{i,max}$, as in option 1; beyond, there is no observation left and the trajectory is extended by increments of the process drawn with the given parameters (not a bridge): the first one starts from the last observation $x_{i,N}$. This needs all the parameters of the process (for instance b for the gamma process, see the

arguments). Under the model, with the true parameters, the pooled sample is also i.i.d. with the law of X_h; but the extended values do not come from the data: they follow the null model by construction and dilute the information of the observations, which lowers the power of a test, all the more as the trajectories have different durations. Use it only for trajectories of similar durations.

When to use "MultiBridge". For several trajectories observed at different times, when one test on all the trajectories together is wanted. With one trajectory, both options give the method "Bridge". The method "ExtBridge" with `common_step = TRUE` also gives a common step, with a grid that contains all the inspection times, but it can have many more grid points.

References

Grall-Maës, E. (2012). Use of the Kolmogorov-Smirnov test for gamma process. *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability*, 226(6), 624-634. doi:10.1177/1748006X12462522

Madan, D. B., Carr, P. P. and Chang, E. C. (1998). The variance gamma process and option pricing. *European Finance Review*, 2(1), 79-105. doi:10.1023/A:1009703431535

Hörmann, W. and Leydold, J. (2014). Generating generalized inverse Gaussian random variates. *Statistics and Computing*, 24, 547-557. doi:10.1007/s1122201393873

See Also

- `increments()`: the reconstructed increments;
- `plot_bridge()`: observed and resampled values;
- `gamma_bridge()`: bridge of the gamma process;
- `wiener_bridge()`: bridge of the Wiener process;
- `ig_bridge()`: bridge of the inverse Gaussian process;
- `meixner_bridge()`: bridge of the Meixner process.

Examples

```
# 1. One trajectory observed at 11 irregular times (theta = 0.5, sigma = 1.5, nu = 2)
t <- c(0, 1.2, 2, 3.8, 5.5, 7.1, 10.4, 12.9, 15.3, 18.7, 22.5)
x <- c(0, 0.9, 0.4, 2.8, 3.1, 5.6, 7.2, 6.5, 9.8, 11.3, 12.1)
set.seed(1)
r <- vg_bridge(x, t, theta = 0.5, sigma = 1.5, nu = 2)
r

# The resampled trajectory, on the regular grid 0, 2.25, 4.5, ..., 22.5
r[[1]]

# Time step of the grid: t_N / N = 22.5 / 10
attr(r, "teq")

# 2. Reconstructed increments, the input of the goodness-of-fit tests
increments(r)

# 3. Observed and resampled values
plot_bridge(r)

# 4. Grid points equal to inspection times keep the observed values:
#    here the grid is 0, 2, 4, 6, 8, 10 and 2 and 6 are inspection times
t2 <- c(0, 2, 3, 6, 7, 10)
x2 <- c(0, 1.4, 0.9, 3.1, 2.6, 4.8)
```

```
vg_bridge(x2, t2, theta = 0.5, sigma = 1.5, nu = 2)[[1]]

# 5. One trajectory given as a data frame (times, then levels)
vg_bridge(data.frame(t = t2, X = x2), theta = 0.5, sigma = 1.5, nu = 2)

# 6. Several trajectories, each with its own inspection times:
#    one regular grid and one time step per trajectory
liste <- list(data.frame(t = c(0, 1, 3, 6), X = c(0, 0.4, 2.3, 1.9)),
              data.frame(t = c(0, 2, 4, 5, 9), X = c(0, 1.6, 0.8, 3.1, 4.2)))
r2 <- vg_bridge(liste, theta = 0.5, sigma = 1.5, nu = 2)
r2
attr(r2, "teq")
increments(r2)
plot_bridge(r2, trajectory = 1:2)

# 7. The bridge values are random: set.seed() makes them reproducible
set.seed(2); y1 <- vg_bridge(x, t, theta = 0.5, sigma = 1.5, nu = 2)[[1]]$X
set.seed(2); y2 <- vg_bridge(x, t, theta = 0.5, sigma = 1.5, nu = 2)[[1]]$X
identical(y1, y2)

# 8. Method "ExtBridge": the times have 1 decimal place, the grid
#    0, 0.1, ..., 22.5 contains all of them
r3 <- vg_bridge(x, t, theta = 0.5, sigma = 1.5, nu = 2, method = "ExtBridge")
attr(r3, "teq")
all(t %in% r3[[1]]$t)
plot_bridge(r3)

# Error: the first time must be 0
try(vg_bridge(x, t + 1, theta = 0.5, sigma = 1.5, nu = 2))

# Method "MultiBridge": several trajectories with their own times on one common grid,
# so that all the reconstructed increments have the same law
liste4 <- list(data.frame(t = c(0, 3, 8), X = c(0, 1.5, 4.2)),
               data.frame(t = c(0, 2, 5, 9, 12), X = c(0, 0.9, 2.8, 4.1, 6.5)),
               data.frame(t = c(0, 4, 7, 11, 15), X = c(0, 2.2, 3.1, 5.6, 7.4)))
r4 <- vg_bridge(liste4, theta = 0.5, sigma = 1.5, nu = 2, method = "MultiBridge")
attr(r4, "teq")
sapply(r4, nrow)

# 9. Ten trajectories simulated with DegradSim, inspected more and more often
plan <- DegradSim::inspection_times("increasing", n = 20, n_traj = 10, tmax = 40)
sims <- DegradSim::sim_vg(plan, theta = 0.5, sigma = 1.5, nu = 2)

# Parameters estimated with levyfit
# With the true parameters (they can be estimated with levyfit::vg_mle())
r <- vg_bridge(sims, theta = 0.5, sigma = 1.5, nu = 2)
r
plot_bridge(r, trajectory = 1:4)

# Reconstructed increments of all the trajectories
d <- increments(r)
head(d)
# Under the model, the increments have mean theta * teq and
# variance (sigma^2 + nu * theta^2) * teq
c(mean(d$dx), 0.5 * d$teq[1], var(d$dx), (1.5^2 + 2 * 0.5^2) * d$teq[1])
```

`wiener_bridge` *Brownian bridge: resampling on a regular grid*

Description

Resamples one or several trajectories of a Wiener process with drift, observed at irregular inspection times, onto a regular time grid, by exact simulation of the Brownian bridge between the known values.

Usage

```
wiener_bridge(
  x,
  t = NULL,
  sigma,
  method = "Bridge",
  digits = NULL,
  common_step = FALSE,
  option = 1,
  mu = NULL
)
```

Arguments

`x`	Degradation levels of one trajectory; or a list of data frames, one per trajectory, whose first column contains the inspection times and second column the degradation levels (for instance the result of a `DegradSim` simulation function).
`t`	Inspection times of the trajectory, starting at 0 and strictly increasing. Not used when `x` is a list.
`sigma`	Volatility $\sigma > 0$ (standard deviation per square root of time unit). The package `levyfit` returns the variance: use `sqrt(wiener_mle(x)[["sigma2"]])`.
`method`	`"Bridge"` (default) for the algorithm of Grall-Maës (2012), with as many grid points as observations and one grid per trajectory; `"ExtBridge"` for its extension, whose grid contains all the inspection times (see the section Method "ExtBridge"); `"MultiBridge"` for several trajectories on one common grid (see the section Method "MultiBridge").
`digits`	Used only with `method = "ExtBridge"`: number of decimal places of the inspection times, which sets the time step of the grid. `NULL` (default) finds it automatically.
`common_step`	Used only with `method = "ExtBridge"` and several trajectories: `FALSE` (default) gives each trajectory its own time step h; `TRUE` gives all the trajectories the same time step, the largest one such that every inspection time of every trajectory is a grid point. With a common step, the reconstructed increments of all the trajectories have the same law under the model and can be tested together.
`option`	Used only with `method = "MultiBridge"`: `1` (default) to resample each trajectory up to its own last inspection, `2` to resample all the trajectories up to the last inspection time over all the trajectories, extending the shorter ones by increments of the process (see the section Method "MultiBridge").
`mu`	Drift of the Wiener process, real; used only by the option 2 of the method "MultiBridge", to extend the trajectories.

Value

An object of class `"levybridge"`: a list with one data frame per trajectory, with columns `t` (regular grid) and `X` (resampled levels). Its attributes are `model`, `parameters`, `method`, `teq` (time step of the grid of each trajectory) and `observed` (the observed data, one data frame per trajectory; with the method "ExtBridge", the times are the rounded ones when a rounding was needed). Use `increments()` for the reconstructed increments and `plot_bridge()` to compare observed and resampled values.

Model

The Wiener process with drift μ and volatility $\sigma > 0$ is $X_t = \mu t + \sigma B_t$, with B a standard Brownian motion: $X_0 = 0$, independent increments and $X_t - X_s \sim \mathcal{N}(\mu(t-s), \sigma^2(t-s))$. Its trajectories are not monotone.

Law of the bridge

Let $t_A < t < t_B$, $\tau_1 = t - t_A$, $\tau_2 = t_B - t$, $T = \tau_1 + \tau_2$ and $\Delta = x_B - x_A$. Given $X_{t_A} = x_A$ and $X_{t_B} = x_B$,

$$X_t \sim N\Big(x_A + \Delta\frac{\tau_1}{T},\ \sigma^2\frac{\tau_1\tau_2}{T}\Big).$$

The law of the bridge does not depend on the drift μ: only σ is needed.

Resampling algorithm (method "Bridge")

This is the interpolation algorithm of Grall-Maës (2012), used for all the processes of the package. Let $0 = t_0 < t_1 < \cdots < t_N$ be the inspection times of a trajectory and $x_0, \ldots, x_N$ the observed levels. The regular grid is $t'_i = i\,\Delta t_{eq}$, $i = 0, \ldots, N$, with $\Delta t_{eq} = t_N/N$: it has as many points as observations and the same end points.

1. $x'_0 = x_0$ and $x'_N = x_N$.
2. For $i = 1, \ldots, N-1$, in increasing order:
   - if t'_i is an inspection time, x'_i is the observed value;
   - otherwise, x'_i is drawn from the bridge between the nearest known point on the left (an inspection or the grid point t'_{i-1} already drawn, whichever is closer) and the nearest inspection on the right.

By the Markov property of the bridge, the vector $(x'_1, \ldots, x'_{N-1})$ follows exactly the conditional law of $(X_{t'_1}, \ldots, X_{t'_{N-1}})$ given the observations. Hence, if the data come from the model and the parameters are the true ones, the reconstructed increments $x'_i - x'_{i-1}$ (see `increments()`) are independent with the law of $X_{\Delta t_{eq}}$: they can be tested with an i.i.d. goodness-of-fit test. A grid point that equals an inspection time up to rounding errors (relative difference below 10^{-9}) is set to that inspection time.

Precision. Each draw splits the increment Δ of the current interval into a left part (up to the grid point) and a right part (up to the next inspection), both computed directly: here ΔY and $\Delta(1-Y)$, with $Y = G_1/(G_1+G_2)$, $G_1 \sim \Gamma(a\tau_1)$, $G_2 \sim \Gamma(a\tau_2)$ drawn on the log scale. The reconstructed increments (`increments()`) are these parts, not differences of levels: a very small increment keeps all its significant digits instead of being rounded to 0, even for very small shapes $a\tau$; only an increment below the smallest positive number (about 10^{-308}), which needs extremely small shapes, is 0.

Method "ExtBridge"

Extension of the method of Grall-Maës whose regular grid contains all the inspection times (Tito and Paroissin). The inspection times are multiples of 10^{-d}, where d is the number of decimal places (argument `digits`), and the time step is the largest one such that every inspection time is a grid point:

$$h = 10^{-d}\,\gcd\bigl(10^d t_1, \ldots, 10^d t_N\bigr), \qquad t'_k = k\,h, \quad k = 0, \ldots, t_N/h.$$

The grid is then covered as in the method "Bridge": every inspection time keeps its observed value, and the other grid points are drawn from the bridge between the last known value on the left and the next inspection. The reconstructed increments are independent with the law of X_h, and the pseudo-sample contains all the observations. The grid has $t_N/h \geq N$ steps, with equality if and only if the inspection times are regular; then the method returns the observations themselves.

With `digits = NULL` (default), d is the smallest number of decimal places, between 0 and 6, such that all the inspection times are multiples of 10^{-d}. If there is none (for instance times drawn with `runif()`), the times are rounded to 2 decimal places, with a warning; give `digits` to choose another rounding. When `digits` is given and some times have more decimal places, they are rounded to `digits` decimal places, with a warning; two times equal after rounding give an error. A warning is also given when the grid has more than 10^5 steps, since the resampling then takes time. The grid and the time step are computed for each trajectory; with `common_step = TRUE`, d and h are computed from the inspection times of all the trajectories, so that all the grids have the same step.

Method "MultiBridge"

Version of the method "Bridge" for several trajectories, with **one regular grid common to all the trajectories** (Tito and Paroissin). With the method "Bridge", each trajectory has its own time step t_N/N: the reconstructed increments of two trajectories then have different laws and cannot be tested together. With the method "MultiBridge", all the reconstructed increments have the law of X_h for the same h: they form a single i.i.d. sample under the model, and one goodness-of-fit test can be applied to all the trajectories together.

Time step. For n trajectories, the trajectory i being observed on $[0, t_{i,max}]$ with m_i increments,

$$h = \frac{t_{1,max} + \cdots + t_{n,max}}{m_1 + \cdots + m_n},$$

the total observed duration divided by the total number of increments. The number of reconstructed increments is then about the number of observed increments, as with the method "Bridge"; for one trajectory, $h = t_N/N$ is the step of the method "Bridge". The grid is $t'_k = kh$; as with the method "Bridge", it does not necessarily contain the inspection times (a grid point equal to an inspection time keeps the observed value).

Option 1 (`option = 1`, default): each trajectory is resampled up to its own last inspection, on the grid points $kh \leq t_{i,max}$. Each grid point is drawn from the bridge between the last known value on the left and the next inspection, exactly as with the method "Bridge". The end of the trajectory, between its last grid point and $t_{i,max}$, is not used. A long trajectory gives many reconstructed increments, a short one few (none when $t_{i,max} < h$, with a warning). Under the model, with the true parameters, all the reconstructed increments are independent with the law of X_h: the pooled sample is exactly i.i.d. This is the recommended option.

Option 2 (`option = 2`): all the trajectories are resampled up to the last inspection time over all the trajectories, $\max_i t_{i,max}$, so that they all have the same number of grid points. Up to $t_{i,max}$, as in option 1; beyond, there is no observation left and the trajectory is extended by increments of the process drawn with the given parameters (not a bridge): the first one starts from the last observation $x_{i,N}$. This needs all the parameters of the process (for instance b for the gamma process, see the

arguments). Under the model, with the true parameters, the pooled sample is also i.i.d. with the law of X_h; but the extended values do not come from the data: they follow the null model by construction and dilute the information of the observations, which lowers the power of a test, all the more as the trajectories have different durations. Use it only for trajectories of similar durations.

When to use "MultiBridge". For several trajectories observed at different times, when one test on all the trajectories together is wanted. With one trajectory, both options give the method "Bridge". The method "ExtBridge" with `common_step = TRUE` also gives a common step, with a grid that contains all the inspection times, but it can have many more grid points.

References

Grall-Maës, E. (2012). Use of the Kolmogorov-Smirnov test for gamma process. *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability*, 226(6), 624-634. doi:10.1177/1748006X12462522

See Also

- `increments()`: the reconstructed increments;
- `plot_bridge()`: observed and resampled values;
- `gamma_bridge()`: bridge of the gamma process;
- `ig_bridge()`: bridge of the inverse Gaussian process;
- `vg_bridge()`: bridge of the variance-gamma process;
- `meixner_bridge()`: bridge of the Meixner process.

Examples

```
# 1. One trajectory observed at 11 irregular times (mu = 0.5, sigma = 1.5)
t <- c(0, 1.2, 2, 3.8, 5.5, 7.1, 10.4, 12.9, 15.3, 18.7, 22.5)
set.seed(1)
x <- c(0, cumsum(0.5 * diff(t) + 1.5 * sqrt(diff(t)) * rnorm(10)))
set.seed(1)
r <- wiener_bridge(x, t, sigma = 1.5)
r

# The resampled trajectory, on the regular grid 0, 2.25, 4.5, ..., 22.5
r[[1]]

# Time step of the grid: t_N / N = 22.5 / 10
attr(r, "teq")

# 2. Reconstructed increments, the input of the goodness-of-fit tests
increments(r)

# 3. Observed and resampled values
plot_bridge(r)

# 4. Grid points equal to inspection times keep the observed values:
#    here the grid is 0, 2, 4, 6, 8, 10 and 2 and 6 are inspection times
t2 <- c(0, 2, 3, 6, 7, 10)
x2 <- c(0, 1.4, 0.9, 3.1, 2.6, 4.8)
wiener_bridge(x2, t2, sigma = 1.5)[[1]]

# 5. One trajectory given as a data frame (times, then levels)
wiener_bridge(data.frame(t = t2, X = x2), sigma = 1.5)
```

```
# 6. Several trajectories, each with its own inspection times:
#    one regular grid and one time step per trajectory
liste <- list(data.frame(t = c(0, 1, 3, 6), X = c(0, 0.4, 2.3, 1.9)),
              data.frame(t = c(0, 2, 4, 5, 9), X = c(0, 1.6, 0.8, 3.1, 4.2)))
r2 <- wiener_bridge(liste, sigma = 1.5)
r2
attr(r2, "teq")
increments(r2)
plot_bridge(r2, trajectory = 1:2)

# 7. The bridge values are random: set.seed() makes them reproducible
set.seed(2); y1 <- wiener_bridge(x, t, sigma = 1.5)[[1]]$X
set.seed(2); y2 <- wiener_bridge(x, t, sigma = 1.5)[[1]]$X
identical(y1, y2)

# 8. Method "ExtBridge": the times have 1 decimal place, the grid
# Method "MultiBridge": several trajectories with their own times on one common grid,
# so that all the reconstructed increments have the same law
liste4 <- list(data.frame(t = c(0, 3, 8), X = c(0, 1.5, 4.2)),
               data.frame(t = c(0, 2, 5, 9, 12), X = c(0, 0.9, 2.8, 4.1, 6.5)),
               data.frame(t = c(0, 4, 7, 11, 15), X = c(0, 2.2, 3.1, 5.6, 7.4)))
r4 <- wiener_bridge(liste4, sigma = 1.5, method = "MultiBridge")
attr(r4, "teq")
sapply(r4, nrow)

#    0, 0.1, ..., 22.5 contains all of them
r3 <- wiener_bridge(x, t, sigma = 1.5, method = "ExtBridge")
attr(r3, "teq")
all(t %in% r3[[1]]$t)
plot_bridge(r3)

# Error: the first time must be 0
try(wiener_bridge(x, t + 1, sigma = 1.5))

# 9. Ten trajectories simulated with DegradSim, inspected more and more often
plan <- DegradSim::inspection_times("increasing", n = 20, n_traj = 10, tmax = 40)
sims <- DegradSim::sim_wiener(plan, mu = 0.5, sigma = 1.5)

# Parameters estimated with levyfit
sigma_hat <- sqrt(levyfit::wiener_mle(sims)["sigma2"])
r <- wiener_bridge(sims, sigma = sigma_hat)
r
plot_bridge(r, trajectory = 1:4)

# Reconstructed increments of all the trajectories
d <- increments(r)
head(d)
# Under the model, the increments follow N(mu * teq, sigma^2 * teq)
ks.test(d$dx, "pnorm", mean = 0.5 * d$teq[1], sd = 1.5 * sqrt(d$teq[1]))
```

Index

∗ **package**
levyBridge-package, 2

gamma_bridge, 5
gamma_bridge(), *2*, *15–17*, *22*, *24*, *28*, *33*
GIGrvg::rgig(), *26*
gsl::lngamma_complex(), *20*

ig_bridge, 11
ig_bridge(), *2*, *9*, *17*, *22*, *28*, *33*
increments, 16
increments(), *3*, *6–8*, *12–14*, *19*, *20*, *22*, *24–26*, *28*, *31*, *33*

levyBridge *(*levyBridge-package*)*, 2
levyBridge-package, 2

meixner_bridge, 18
meixner_bridge(), *2*, *9*, *15*, *17*, *28*, *33*

plot_bridge, 23
plot_bridge(), *3*, *6*, *8*, *12*, *15*, *19*, *22*, *25*, *28*, *31*, *33*

vg_bridge, 24
vg_bridge(), *2*, *9*, *15*, *17*, *22*, *33*

wiener_bridge, 30
wiener_bridge(), *2*, *8*, *15*, *17*, *22*, *28*